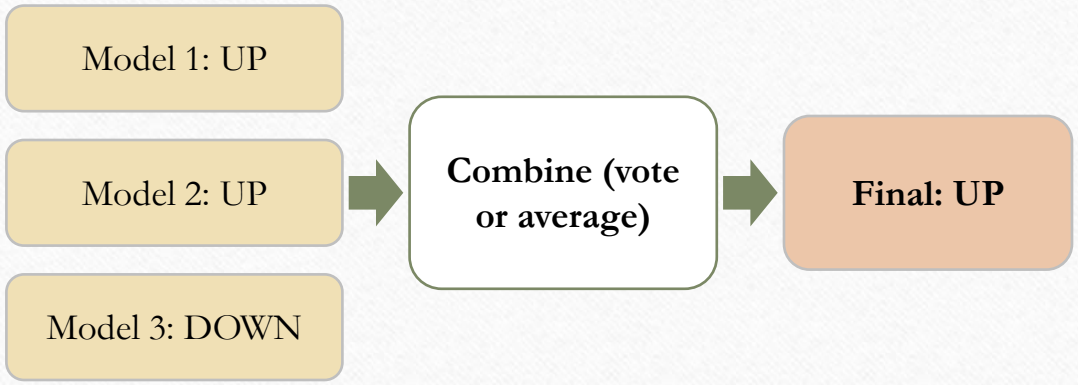


Machine Learning LAB

Ensemble Models

WHAT IS ENSEMBLE LEARNING

- Train several models on the same task and combine their predictions
- The combined answer is often better than the best single model
- Wisdom of the crowd: ask many people and average the answers
- Works for classification, regression and anomaly detection
- Random forest is already an ensemble of decision trees



Majority vote of three classifiers

THREE GROUPS OF ENSEMBLES

Averaging methods

- Voting, bagging, random forests, extra trees
- Models are trained independently
- Predictions are voted or averaged

Boosting methods

- Gradient Boosting, AdaBoost, XGBoost
- Models are trained one after another
- Each model focuses on the mistakes of the previous ones

Stacking

- Several different models make predictions
- A final model learns how to combine them
- The final model is trained on their predictions

Most of these are in scikit-learn. XGBoost is a separate package, and stacking is available as StackingClassifier.

IMPORTS

```
import numpy as np
import pandas as pd
import seaborn as sns
import sklearn
import matplotlib.pyplot as plt
%matplotlib inline

import warnings
warnings.filterwarnings('ignore')
```

ELECTRICITY DATASET

45,312	8	2
instances	features	classes: UP, DOWN

- Source: OpenML electricity (id 151), New South Wales electricity market in Australia
- Each row is a 30-minute period, so there are 48 rows per day
- Dates run from 7 May 1996 to 5 December 1998
- Target: did the price go UP or DOWN compared with the moving average of the last 24 hours

DATASET FEATURES

Feature	Meaning
`date`	Date from 7 May 1996 to 5 December 1998, scaled to 0 to 1
`day`	Day of the week, 1 to 7 (Monday to Sunday)
`period`	Half-hour slot of the day, 1 to 48, scaled to 0 to 1
`nswprice`	New South Wales electricity price, scaled
`nswdemand`	New South Wales electricity demand, scaled
`vicprice`	Victoria electricity price, scaled
`vicdemand`	Victoria electricity demand, scaled
`transfer`	Scheduled electricity transfer between the two states, scaled
`class`	Target: price goes UP or DOWN

LOADING THE DATA

```
from sklearn.datasets import fetch_openml

elec_data = fetch_openml(name='electricity', version=1)
elec_df = elec_data.frame

print("Data shape:", elec_data.data.shape)
print("Feature names:", elec_data.feature_names)
print("Target names:", elec_data.target_names)
```

```
Data shape: (45312, 8)
Feature names: ['date', 'day', 'period', 'nswprice', 'nswdemand', 'vicprice', 'vicdemand', 'transfer']
Target names: ['class']
```

SPLITTING THE DATA

```
from sklearn.model_selection import train_test_split

train_data, test_data = train_test_split(
    elec_df, test_size=0.25, random_state=20
)

print("Training samples:", len(train_data))
print("Testing samples:", len(test_data))
```

```
Training samples: 33984
Testing samples: 11328
```

We split before exploring, so the test set stays untouched until the final evaluation.

QUICK LOOK AT THE DATA

```
train_data.head(10)
```

	date	day	period	nswprice	nswdemand	vicprice	vicdemand	transfer	class
27325	0.469846	4	0.276596	0.164705	0.519637	0.011417	0.657949	0.265789	DOWN
28731	0.474227	5	0.574468	0.024919	0.191907	0.001656	0.090886	0.819737	DOWN
8450	0.023141	3	0.042553	0.065270	0.250074	0.003467	0.422915	0.414912	DOWN
36659	0.889385	2	0.744681	0.148193	0.670039	0.009981	0.533402	0.563596	UP
781	0.000708	4	0.276596	0.124204	0.475454	0.003467	0.422915	0.414912	UP
13013	0.428963	7	0.106383	0.055242	0.084647	0.003467	0.422915	0.414912	DOWN
3330	0.009203	1	0.382979	0.045635	0.741892	0.003467	0.422915	0.414912	DOWN
18851	0.446662	2	0.744681	0.183409	0.785034	0.012154	0.757639	0.517105	UP
14838	0.433830	3	0.127660	0.047886	0.141476	0.003467	0.422915	0.414912	DOWN
30462	0.868236	6	0.638298	0.030833	0.702023	0.001963	0.538322	0.674123	UP

DATASET INFORMATION

```
train_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 33984 entries, 27325 to 27098
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   date        33984 non-null  float64
 1   day         33984 non-null  category
 2   period      33984 non-null  float64
 3   nswprice    33984 non-null  float64
 4   nswdemand   33984 non-null  float64
 5   vicprice    33984 non-null  float64
 6   vicdemand   33984 non-null  float64
 7   transfer    33984 non-null  float64
 8   class       33984 non-null  category
dtypes: category(2), float64(7)
memory usage: 2.1 MB
```

- `class` is categorical, so it must be encoded
- All numerical features are already scaled to the range 0 to 1
- `day` is stored as a category and is not scaled

SUMMARY STATISTICS

```
train_data.describe()
```

	date	period	nswprice	nswdemand	vicprice	vicdemand	transfer
count	33984.000000	33984.000000	33984.000000	33984.000000	33984.000000	33984.000000	33984.000000
mean	0.498150	0.499211	0.057922	0.424763	0.003445	0.423035	0.500089
std	0.340429	0.294571	0.040195	0.163858	0.008846	0.121087	0.153224
min	0.000000	0.000000	0.000000	0.001190	0.000000	0.000000	0.000000
25%	0.031857	0.234043	0.035247	0.307944	0.002283	0.372087	0.414912
50%	0.456307	0.489362	0.048667	0.442725	0.003467	0.422915	0.414912
75%	0.880581	0.744681	0.074276	0.535704	0.003467	0.469446	0.605263
max	1.000000	1.000000	0.981806	1.000000	1.000000	1.000000	1.000000

nswprice and vicprice are very skewed: medians of 0.049 and 0.003, but maximums of 0.98 and 1.0.

MISSING VALUES

```
train_data.isnull().sum()
```

```
date          0
day           0
period        0
nswprice      0
nswdemand     0
vicprice      0
vicdemand     0
transfer      0
class         0
dtype: int64
```

If values were missing, you could:

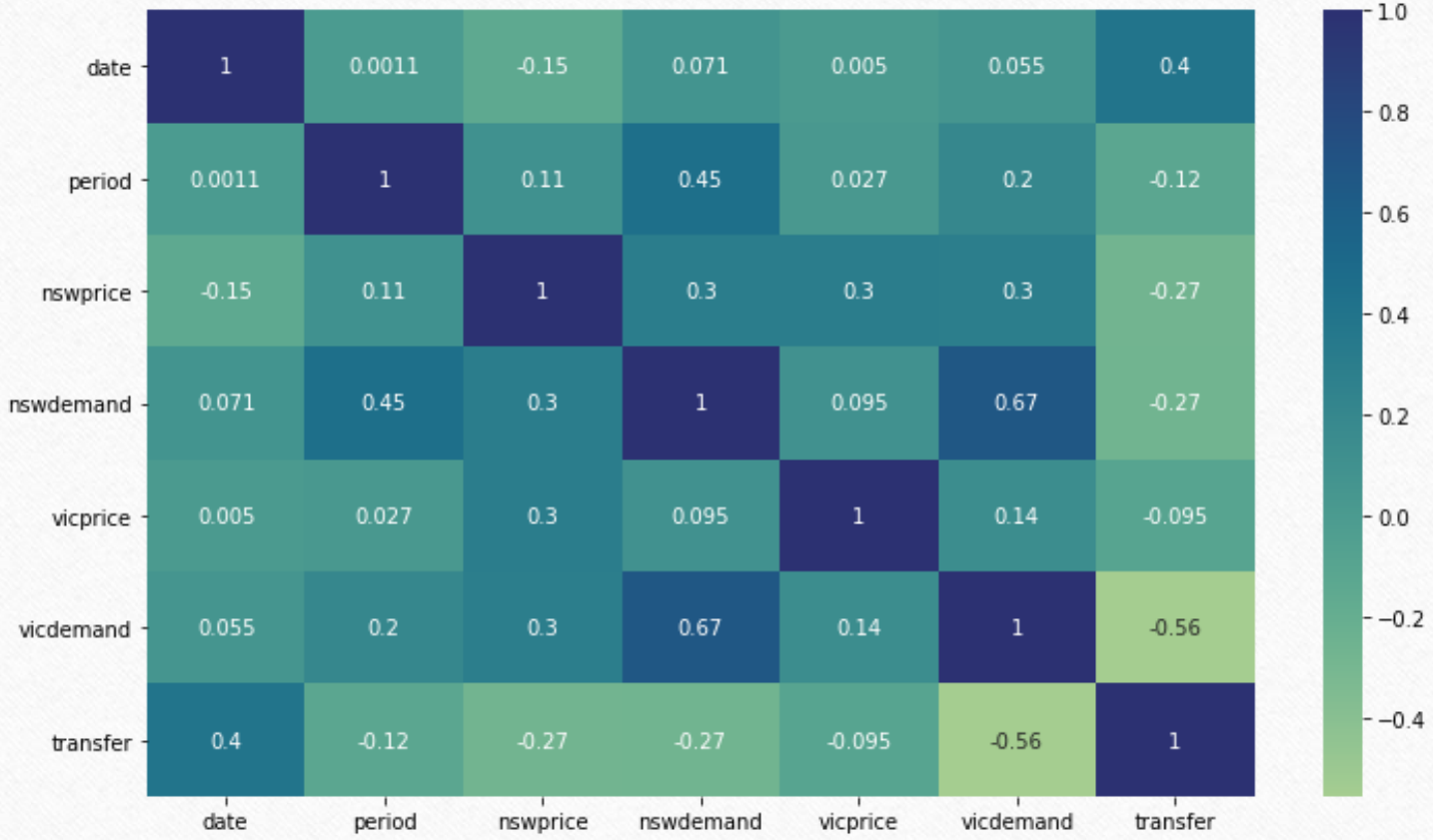
- Remove the rows (you lose data)
- Leave them as they are
- Fill them with the mean, median or most frequent value (you add some noise)
- No option is always best. It depends on the data size and the goal.

CORRELATION HEATMAP

```
corr = train_data.corr(numeric_only=True)

plt.figure(figsize=(12, 7))
sns.heatmap(corr, annot=True, cmap='crest')
plt.show()
```

FEATURE CORRELATION

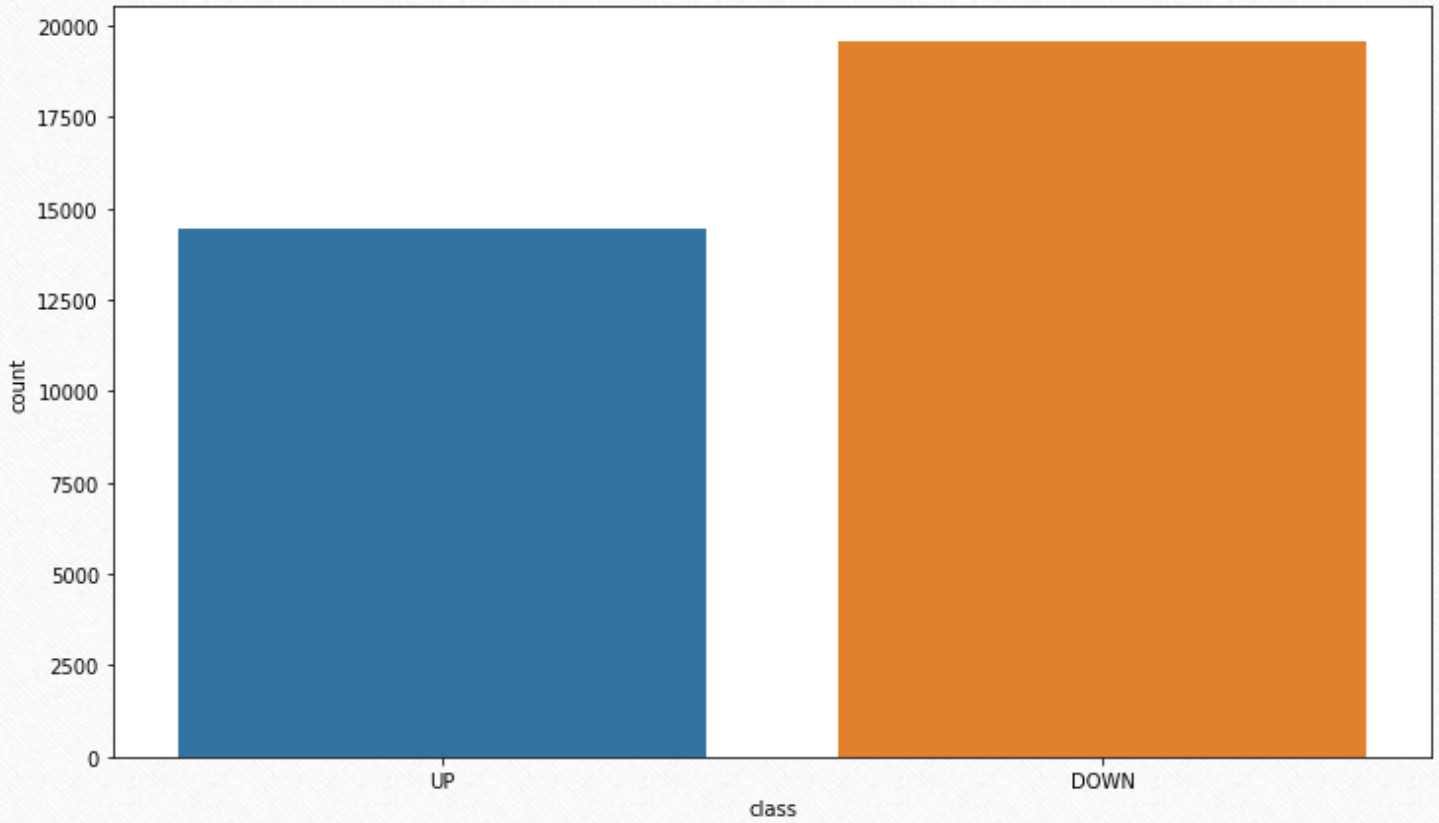


- No pair of features is strongly correlated
- Strongest pairs: nswdemand and vicdemand (0.67), vicdemand and transfer (−0.56)
- Values near +1 or −1 mean two features carry similar information
- `class` is missing because it is still text

CLASS DISTRIBUTION

```
plt.figure(figsize=(12, 7))
sns.countplot(data=train_data, x='class')
plt.show()
```

UP VERSUS DOWN

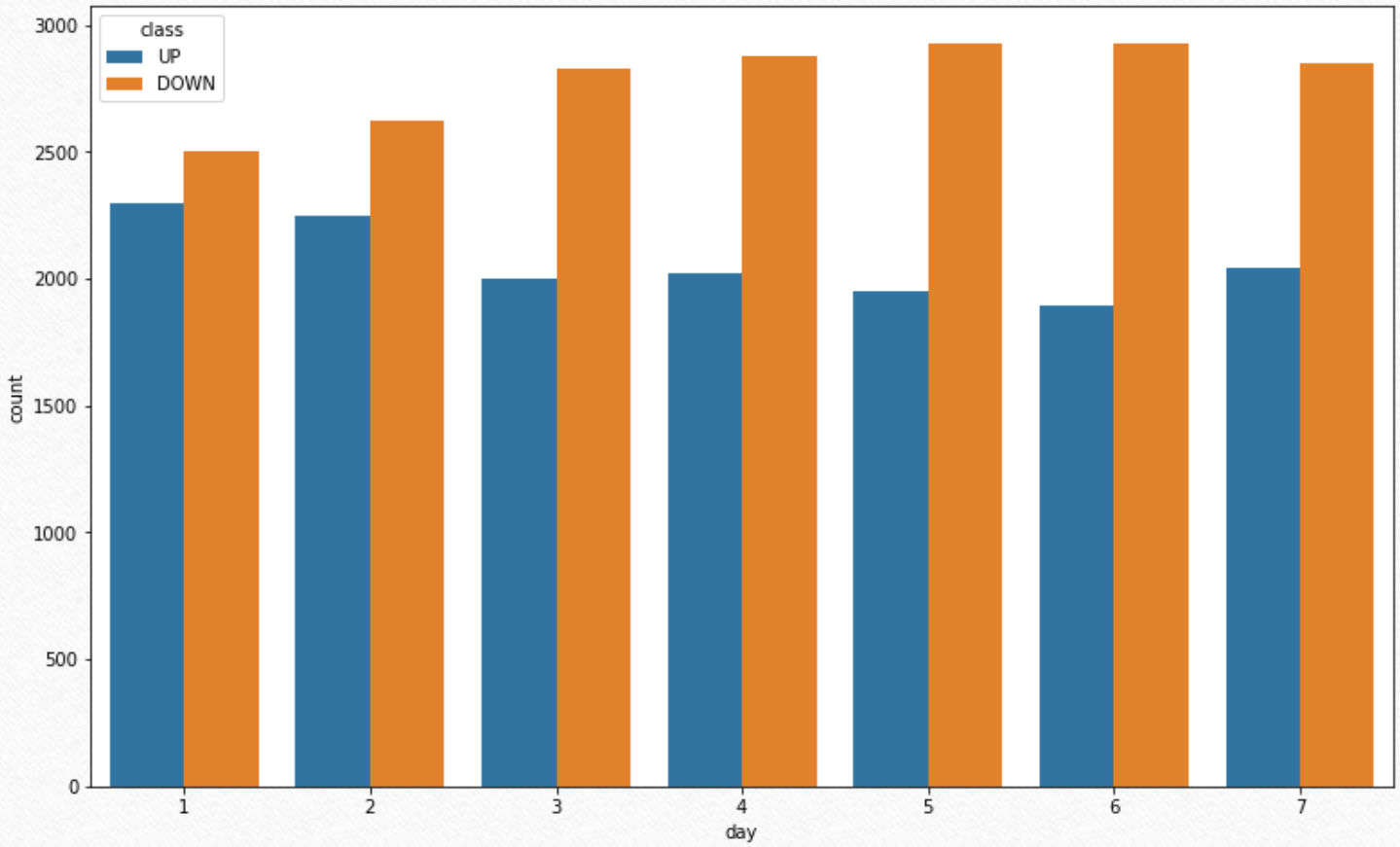


- DOWN is more common than UP
- Roughly 19,500 DOWN rows against 14,500 UP rows in the training data
- The classes are mildly imbalanced, so accuracy alone can hide a bias toward DOWN

CLASS BY DAY OF WEEK

```
plt.figure(figsize=(13, 8))
sns.countplot(data=train_data, x='day', hue='class')
plt.show()
```

UP AND DOWN PER DAY

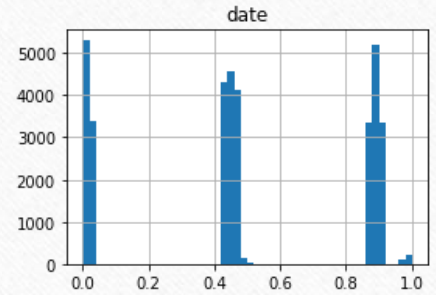


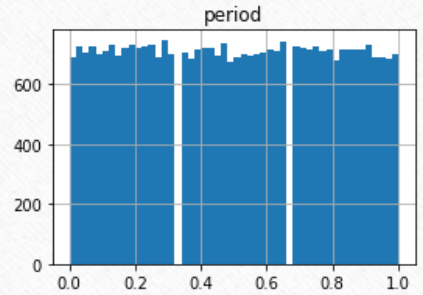
- DOWN outnumbers UP on every day
- The gap is small on day 1 (Monday) and widest on days 5 and 6
- `day` carries some information about the class

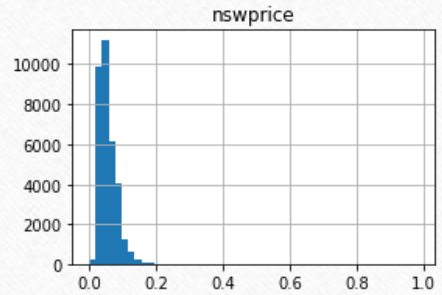
FEATURE HISTOGRAMS

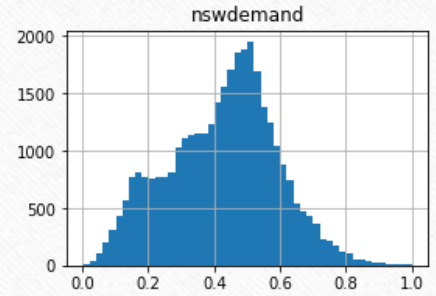
```
train_data.hist(bins=50, figsize=(15, 10))
plt.show()
```

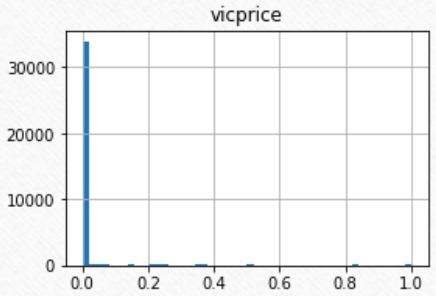
HISTOGRAMS

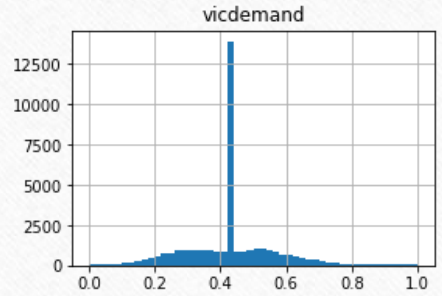


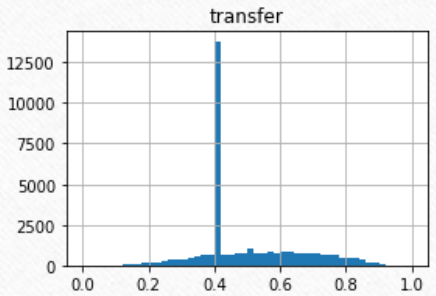












- nswprice and vicprice are concentrated near 0
- vicdemand and transfer have a large spike at one repeated value
- nswdemand is roughly bell shaped, and period is uniform
- date shows three separate clusters

PREPROCESSING

```
from sklearn.preprocessing import LabelEncoder

X_train = train_data.drop('class', axis=1)
y_train = train_data['class']

label_enc = LabelEncoder()
y_train_prepared = label_enc.fit_transform(y_train)

print(label_enc.classes_)
```

```
['DOWN' 'UP']
```

DOWN becomes 0 and UP becomes 1. Tree based models do not need feature scaling, so day is left as it is.

ACCURACY HELPER

```
from sklearn.metrics import accuracy_score

def accuracy(model, data, labels):
    predictions = model.predict(data)
    acc = accuracy_score(labels, predictions)

    return acc
```

We reuse this small function to score every ensemble below.

VOTING CLASSIFIER

Hard voting

- Each model predicts a class label
- The majority label wins
- Example: UP, UP, DOWN gives UP

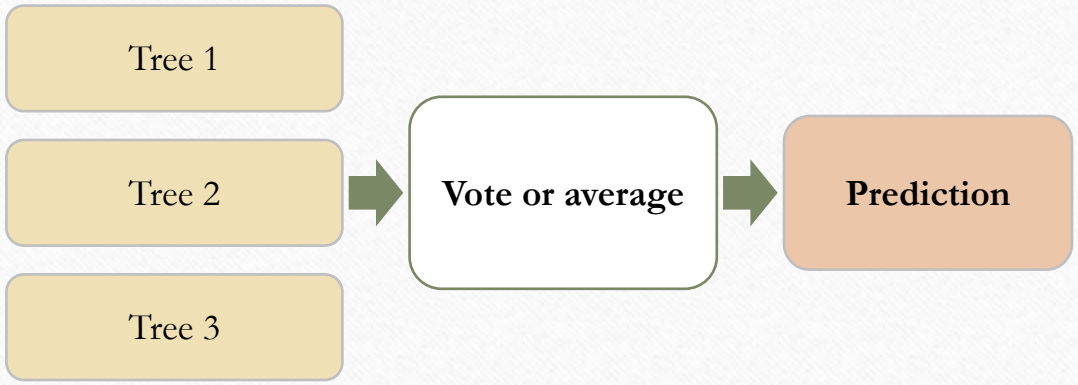
Soft voting

- Average the predicted class probabilities
- The class with the highest average wins
- Example: P(UP) = 0.55, 0.55, 0.05 averages 0.38, so DOWN

- Soft voting needs models with `predict_proba`. For SVC, set `probability=True`.
- Different model types make different mistakes, which is why voting can help

BAGGING

- Same algorithm, trained on different random subsets of the data
- Predictions are combined by voting or averaging
- Works well with decision trees, which overfit easily: averaging reduces the variance
- `bootstrap=True` samples with replacement (bagging)
- `bootstrap=False` samples without replacement (pasting)



Each tree is trained on a different random subset

BAGGING CLASSIFIER

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_classifier = BaggingClassifier(
    DecisionTreeClassifier(class_weight='balanced'),
    max_samples=0.5, max_features=0.5, bootstrap=False
)

bag_classifier.fit(X_train, y_train_prepared)
print(accuracy(bag_classifier, X_train, y_train_prepared))
```

```
0.986670197740113
```

Each tree sees 50% of the rows and 50% of the features. n_estimators is not set, so the default of 10 trees is used. This is training accuracy.

GRADIENT BOOSTING

- Trees are trained in sequence, not independently
- Each new tree tries to correct the errors of the trees before it
- Key settings: `n_estimators` and `learning_rate`
- A high learning rate needs fewer trees. A low one needs more trees but usually generalises better
- Can overfit if pushed too hard



GRADIENT BOOSTING CODE

```
from sklearn.ensemble import GradientBoostingClassifier

grad_boost_clf = GradientBoostingClassifier(
    n_estimators=500,
    learning_rate=0.8,
    random_state=42,
    max_depth=2
)

grad_boost_clf.fit(X_train, y_train_prepared)
print(accuracy(grad_boost_clf, X_train, y_train_prepared))
```

```
0.9269362052730696
```

learning_rate=0.8 is fairly high. Lab idea: try 0.1 and compare train and test accuracy.

ADABOOST

- Also a boosting method, but it re-weights the training samples
- Samples that a tree gets wrong receive a higher weight for the next tree
- The final prediction is a weighted vote of all the trees
- Main settings: `n_estimators`, `learning_rate` and the depth of each tree



ADABOOST CODE

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

adaboost_clf = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=3, class_weight='balanced'),
    n_estimators=300,
    learning_rate=0.5
)

adaboost_clf.fit(X_train, y_train_prepared)
print(accuracy(adaboost_clf, X_train, y_train_prepared))
```

```
0.93717631826742
```

estimator was called base_estimator before scikit-learn 1.2. The score above comes from the original run and can differ slightly across versions.

XGBOOST

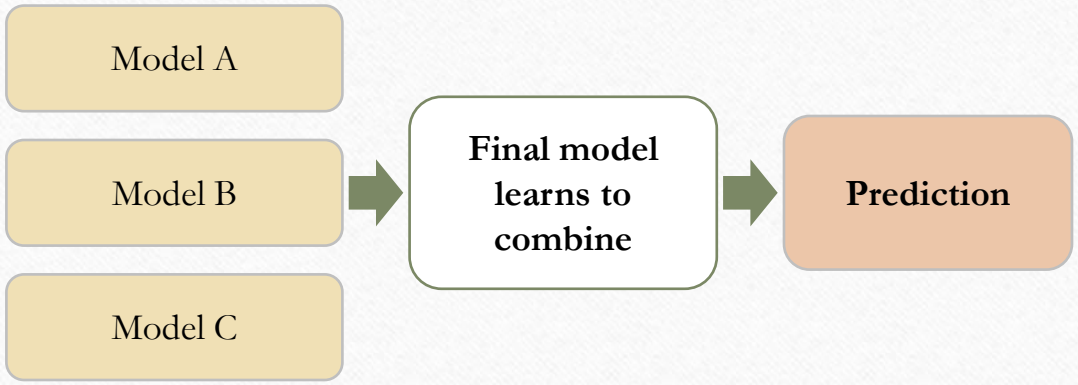
- eXtreme Gradient Boosting: an optimised, fast version of gradient boosting
- Not part of scikit-learn, but it follows the same fit and predict style
- Install with `pip install xgboost`

```
import xgboost as xgb

xgb_clf = xgb.XGBClassifier()
xgb_clf.fit(X_train, y_train_prepared)
```

STACKING

- Several different models are trained on the same training set
- A final model, the meta-learner, learns how to combine their predictions
- It is trained on cross-validated (out-of-fold) predictions to avoid leakage
- In scikit-learn: `StackingClassifier`



EVALUATING ON TEST SET

```
X_test = test_data.drop('class', axis=1)
y_test = test_data['class']

y_test_prepared = label_enc.transform(y_test)

print(accuracy(grad_boost_clf, X_test, y_test_prepared))
print(accuracy(bag_classifier, X_test, y_test_prepared))
```

```
0.8968926553672316
0.899364406779661
```

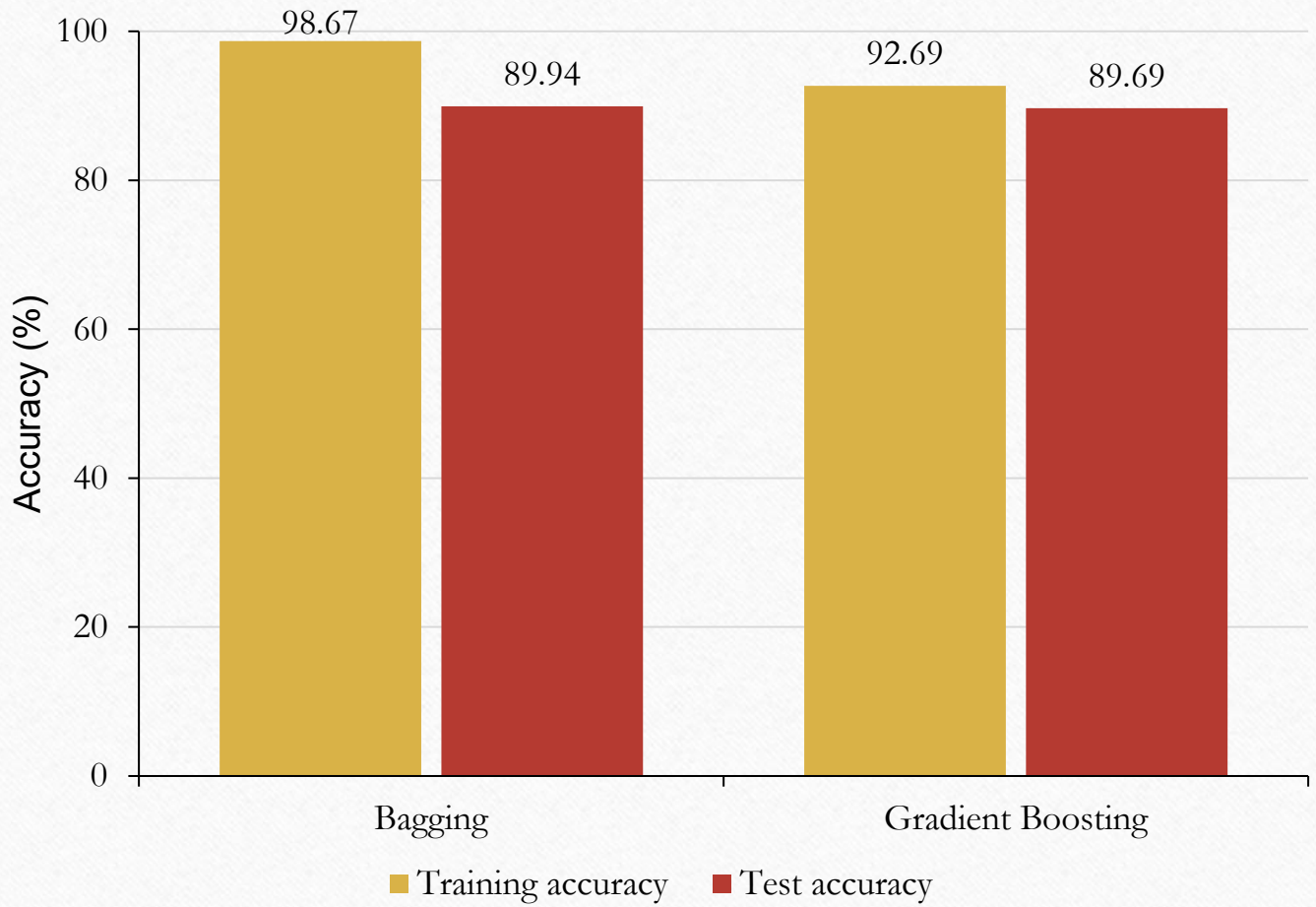
The test set is prepared like the training set. Use transform here, not fit_transform.

TEST SET RESULTS

Model	Training accuracy	Test accuracy	Gap
Bagging	98.67%	89.94%	8.73 points
Gradient Boosting	92.69%	89.69%	3.00 points
AdaBoost	93.72%	not evaluated	-
Stacking	not reported	90.22%	-

- The three test scores are within about 0.5 points of each other (89.7% to 90.2%)
- With 11,328 test rows, the standard error is about 0.3 points, so treat them as roughly equal
- Bagging overfits the most: 98.7% on training data, 89.9% on test data

TRAIN VERSUS TEST



- Training accuracy overstates how good a model is
- Bagging loses almost 9 points on unseen data, gradient boosting about 3
- After that drop, both land at nearly the same test score
- Always compare models on the test set

FINAL NOTES

- Combining models often beats a single model: the wisdom of the crowd
- Bagging mainly reduces variance, boosting mainly reduces bias, stacking learns the combination
- Judge models on the test set, not on training accuracy
- To improve results, start with better data, then tune hyperparameters
- Several small models can replace one large complex model

Think about it

- The rows are ordered in time. Would a split by date give a lower test accuracy than a random split?
- Try `learning_rate=0.1` in Gradient Boosting and compare train and test accuracy.
- Further reading: Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, Chapter 7